

Multicore Processors Final Report: AST-Flat Trees

Alex Lee  
New York University  
April 2025

## **Abstract**

Efficient querying and organization of multi-dimensional data is a fundamental challenge in domains such as robotics, computer vision, and real-time systems, where fast response times and high data throughput are critical. K-dimensional trees (KD-trees) are a commonly used data structure used for this purpose, enabling operations like nearest neighbor and range searches. However, traditional KD-tree implementations are inherently sequential and begin to break down in high-dimensional spaces due to the curse of dimensionality. Moreover, they are not optimized for modern multi-core processors or for workloads requiring dynamic updates. As applications grow in scale and complexity, there is a need for KD-tree variants that can handle large datasets efficiently, scale with hardware concurrency, and maintain fast response times even in high dimensions or dynamic settings.

To address these challenges, this project proposes a hybrid KD-tree design—AST-Flat KD-tree—that combines random projection-based dimensionality reduction with a parallel, flattened recursive construction method. By operating in a reduced subspace and leveraging OpenMP-based task parallelism, the structure achieves scalability while retaining support for dynamic updates.

Experimental results show that the AST-Flat KD-tree significantly outperforms traditional sequential and parallel KD-trees in high-dimensional, large-scale k-NN and range queries. It maintains comparable accuracy at the cost of slightly increasing time at smaller values.

## **Introduction**

K-dimensional trees (KD-trees) are widely used data structures for organizing multi-dimensional data, with applications spanning robotics, computer vision, databases, and more. They support critical operations such as nearest neighbor search, range search, insertion, and deletion — all essential in time-sensitive and data-intensive systems. In robotics and real-time systems (RTOS), efficient spatial querying is vital for tasks like collision avoidance, motion planning, simultaneous localization and mapping (SLAM), and sensory fusion.

This report focuses on developing a concurrent KD-tree that supports the key operations — insertion, deletion, range search, nearest neighbor (NN), and k-nearest neighbors (k-NN) — while optimizing for parallel execution. The goal is to enable these operations to scale across multiple threads and with increasing data size and dimensionality, making the data structure suitable for real-time applications with stringent latency and performance requirements. For instance in hard real time tasks like collision avoidance and sensor fusion, the maximum points is 1000-5000 and in 3-6D. In soft real time tasks like SLAM, the points range from 10,000 to 50,000 and 6-10D while for offline processing it can range from 50,000-500,000+ points and 10-20D for map building.

Therefore, there are practical reasons for expanding KD-trees beyond small numbers of points. However, traditional KD-tree implementations are sequential and don't exploit modern multi-core processors effectively. Furthermore, above 10 dimensions, KD-trees start breaking down due to the curse of dimensionality as because KD-trees are cut along a dimension/axis in each split, the more dimensions there are the more branches need to be visited as in lower-dimensional spaces. As robotic systems become more complex and data-heavy, there's a need for parallel and concurrent data structures that are both fast and correct.

To address the dual challenge of high-dimensional data and performance, the method here is to use a hybrid strategy of 2 techniques: flat parallel construction and adaptive subspace technique (AST). The flattened recursive construction allows the tree to be built through a median-based recursive partitioning method so subtrees at higher levels of the tree can be parallelized and enables balanced trees. The adaptive subspace technique combats the curse of dimensionality by leveraging a random projection-based dimensionality reduction technique. Rather than using the full n-dimensional space, the input point is projected into a lower dimensional subspace and all operations operate over the projected space while retaining access to the full-dimensional coordinates for final accuracy. Then for high workload usages, we can apply OpenMP with batched range search and batched k-NN to parallelize these common operations.

## **Literature Review**

The problem of efficiently managing and querying KD-trees under high-dimensionality, parallelism, and dynamic updates has been addressed by various authors. Existing work on KD-trees can be broadly categorized into 2 classes: acceleration (improving the speed of queries) and accuracy (improving ability to query in high dimensional space) methods. These categories each offer important insights and trade-offs but fall short of addressing all the challenges together. The AST-Flat KD-tree, aims to unify the strengths of these approaches while specifically addressing high-dimensional and parallelizable KD-tree usage.

### **Literature Review (Acceleration)**

In the first line of work on parallel construction of KD-trees, Sabrina Shahnaj's thesis [8] tackled the construction of KD-trees in a distributed memory system, motivated by organizing data for astrophysics simulations (e.g., N-body force calculation). Two strategies were tested: the Parallel Median Method: compute a distributed median and recursively subdivide and the Parallel Sorting Method: globally sort before partitioning. This approach demonstrated good scalability to 512 processors and showed significant speedups on large datasets (64M points). Some pros were that they were capable of scaling to large datasets on clusters and used an effective partitioning method, however it focused on 3D point data and had limited applicability to shared memory environments.

In contrast, modern shared-memory approaches—such as AST-Flat KD-tree—utilize OpenMP task parallelism and flattened recursive builds. Instead of global sorting, median partitioning is used with quickselect to reduce overhead and recursively divide the data in memory-efficient layouts. This achieves task-level concurrency suitable for multicore machines and embedded systems.

Another key line of work aims to maintain tree balance while supporting frequent insertions, deletions, and queries. BDL-tree : Batch-Dynamic Logarithmic KD-tree [5] introduces a logarithmic forest structure combining buffer trees and static subtrees. It supports batch insertions using buffer consolidation and vEB-style layout, batch deletions with subtree collapse and rebalance and batched k-NN using shared k-NN buffers and data-parallel traversal. It achieves strong theoretical bounds, has excellent scalability (up to 36+ threads) and has numerous implementation optimizations like bloom filters for deletion. However, it is not designed for high-dimensional spaces and query cost increases due to multiple static trees.

Skeleton-Based Batch Insertion [7] is another route for updating KD-trees efficiently as new data arrives or old data is removed. This approach introduces a batch update algorithm using a tree skeleton, where, A shallow top-half tree is fetched (with depth  $\lambda$ ), input points are sieved into corresponding buckets, each bucket is inserted in parallel, and only unbalanced subtrees are selectively rebuilt. This technique avoids the cost of reconstructing the entire KD-tree for each insertion, offering a hybrid between full rebuilds and naive sequential inserts.

Pros include efficient reuse of existing tree structure, parallel-friendly (disjoint subtrees updated concurrently) and good cache locality and practical performance for moderately dynamic workloads. However, this work fails to address the other usages of KD-trees such as deletion, k-NN and is focused on insertion only.

While these works address aspects of the problem where Shahnaj handles parallel static build, the batch insertion handles fast partial updates, and the BDL-tree handles dynamic workloads, none of them integrate all of the following augmentations for all of the standard KD-Tree usages such as parallel shared-memory construction and querying, combative methods for high dimensional data, and support for batched operations.

### **Literature Review (Accuracy)**

Efficient and accurate querying in high-dimensional spaces is a core challenge for many spatial data structures, particularly the KD-tree and its variants. As dimensionality increases, traditional KD-tree approaches suffer from high query failure rates and degraded performance due to poor partitioning behavior in large or sparse datasets. These papers aim to improve the performance in high dimensional space for accurate retrieval of points.

Panigrahy [3] proposes a modification to the traditional KD-tree search algorithm to improve the success probability in finding the true nearest neighbor in high-dimensional settings. The core idea is to perturb the query point multiple times with Gaussian noise and re-run standard KD-tree queries. This method allows the search to access nearby leaf regions that might contain the true nearest neighbor, thereby avoiding partitions that might exclude close points. The pros are that there are no structural modifications to the tree and it's lightweight while boosting accuracy with minimal overhead. However, there is a trade-off between accuracy and query runtime due to multiple iterations and can end up slower than a traditional query.

Zhang & Zhang [2] address the issue of high-dimensional image features (e.g., 128D SIFT vectors) by using coarse-to-fine hierarchical indexing. They first apply a LSH (Locality Sensitive Hashing) to get a rough location estimate, then refine the search using a position-enhanced KD-tree. The KD-tree is modified to alternate between X and Y image coordinates during the initial levels of the tree. This ensures that the location of the original image influences partitioning in early layers. This hybrid strategy combines global location context with local KD-tree precision, which narrows the search space and boosts accuracy. However, this is tailored for images and image retrieval use cases and while the binning can be used for something like pruning, this work is not easily generalizable.

This last use case is the RKD-Tree [1] which is a hybrid data structure that incorporates ideas from both KD-trees and vantage-point trees (vp-trees) to enhance the accuracy and efficiency of n-nearest neighbor (n-NN) queries in large, high-dimensional point datasets. While KD-trees split data using axis-aligned hyperplanes and vp-trees use distance-based partitions, rkd-trees adopt spatial partitioning like KD-trees but integrate distance-aware pruning mechanisms from vp-trees. Each internal node in the rkd-tree stores an additional radius value which surrounds the subtree in a bounding sphere which can be used along with the triangle inequality to find if a subtree contains a closer point. The rkd-tree outperforms the KD-tree by significantly reducing false-positive traversals into subtrees that cannot contain better neighbors. This leads to fewer distance

computations and offers stable performance in high dimensional space. However, this is optimized for static datasets and there is a larger memory overhead.

Despite all these improvements, Pnigraphy's method assumes multiple attempts are feasible in time-sensitive applications and Zhang & Zhang's solution cannot generalize to unstructured domains. RKD-Tree is closest to the AST-flat tree that is proposed where both optimize patterns through flattening and try to prune better through geometric reasoning at each node. However, AST-flat can incorporate batching and handles insertions and deletions which doesn't require rebuilding the tree. Because RKD-tree improves performance modestly in high dimensions and the pruning degrades as the triangle inequality doesn't take effect (the bounding spheres become larger due to curse of dimensionality), AST-flat which uses projects allows it to effectively reduce even in much higher dimensionality.

## **Proposed Idea**

The objective of this project is to design and implement a high-performance KD-tree variant—named AST-Flat KD-Tree—that supports both dynamic updates and fast k-nearest neighbor (k-NN) and range queries over high-dimensional datasets. This design uses three innovations: (1) subspace projection to reduce dimensional complexity, (2) OpenMP-based parallelism for tree construction and batched search, and (3) support for dynamic insertions and deletions, differentiating it from purely static spatial structures.

Traditional KD-trees degrade in performance in high-dimensional spaces due to the "curse of dimensionality." Furthermore, most are static, making them unsuitable for datasets that evolve over time. To address these challenges the project aims to reduce dimensionality through projection, use parallelism in construction and search, support dynamic operations like insertion/deletion, and optimize common batched functions for faster queries.

### **Subspace Projection for Dimensionality Reduction**

To handle high-dimensional data (e.g.,  $DIM > 20$ ), the original space is projected onto a lower-dimensional subspace ( $PROJ\_DIM = 10$ ) using a random projection matrix. This approximation allows the tree to be traversed in a reduced space while preserving relative distances, which is sufficient for approximate nearest neighbor search. The projection matrix is fixed once per dataset and is shared across all operations.

At runtime, generate a random projection matrix:

```
projection_matrix[PROJ_DIM][MAX_DIM]
```

Each point is projected during both insertion and query operations:

```
proj[i] += projection_matrix[i][j] * orig[j];
```

### **Flat, Balanced Tree Construction Using Quickselect**

Inspired by the median split, use `quickselect()` to choose the median directly on the projected coordinate of the current axis. The `build_kdtree_recursive` function recursively builds left and right subtrees around the median split. This allows for speed up tree construction across CPU cores while maintaining correct structure. Within the recursive code, parallel tasks are spawned up to a safe recursion depth

```
if (depth < 8) {
```

```
    #pragma omp task shared(node)
```

```
    node->left = build_kdtree_recursive(...);
```

```

#pragma omp task shared(node)

node->right = build_kdtree_recursive(...);

#pragma omp taskwait

}

```

Early tree levels process large amounts of data which is ideal for parallelism and as we enter more depth, the leaf partitions become smaller which means there is less need for parallelization. Therefore, we limit to depth 8 to avoid task overhead for small leaf partitions.

#pragma omp task creates lightweight, non-blocking threads that execute the left and right subtree builds concurrently.

#pragma omp taskwait ensures both children are completed before returning up the tree.

The outer #pragma omp parallel + #pragma omp single nowait wraps the initial call to ensure the task tree is rooted correctly.

### **Dynamic Insertions & Deletions with Projection Awareness**

To support evolving datasets without needing to rebuild the tree, insert() inserts using projected coordinates like in standard KD-tree logic

```
if (proj_point[axis] < root->projected[axis]) ...
```

delete\_node() recursively finds the matching node, replacing the subtree minimum along the current axis then recursively re-inserts that point to preserve ordering within the tree. Unlike balanced trees, there is no auto-balancing and insert/delete order can affect the depth but this is mitigated by using projected coordinates to compress.

### **Fast Batched Queries with Cache-Efficient Traversal**

To enable massively parallel nearest neighbor or range queries while maintaining good cache locality and reducing memory contention is relatively simple. Each query runs independently and #pragma omp parallel for distributes queries across threads. Stack allocations (double proj[], HeapNode heap[]) ensure that each thread works in its own memory space so there is no false sharing. Additionally, each query is from the read-only tree so no locks are needed in order to prevent collisions.

In the batched\_range\_search, we keep track of a total\_found shared accumulator updated by all threads so we use reduction(+:total\_found)



```

int total_found = 0;

#pragma omp parallel for reduction(+:total_found)

for (int i = 0; i < num_queries; i++) {

    ...

    total_found += range_search(...);

}

```

Total\_found can have a race condition if multiple threads simultaneously add to it so reduction creates a private copy that accumulates locally then combines all values at the end.

In batched\_knn\_search, each thread has its own index and there are no shared variables being updated so only #pragma omp parallel for is used.

```

#pragma omp parallel for

for (int i = 0; i < num_queries; i++) {

    ...

    avg_dists_out[i] = ...

}

```

## **Experimental Setup**

All of the following experiments and benchmarks will be run through the tests in the given code on the crunchy1 NYU Linux system. On the machine there are 4 AMD Opteron 6272 (2.1 GHz) CPUs to total 64 cores via threading. There is 256 GB of RAM and the OS is Red Hat Enterprise Linux 9.

The C standard is being used by the compiler is given by `#define __STDC_VERSION__ 201710L` or C17 with gcc version 11.5.0.

To benchmark, multiple variables are explored in 4 different tests that test computation, communication and memory intensive usages of the KD-Tree. For all of these, 3 different versions are compared, that being the sequential version of a traditional KD-tree, a parallelized version of a KD-tree and then the AST-Flat KD-Tree which is both augmented and parallelized.

In test 1, batched k-NN queries are run and the variables tested are number of k-NN queries with constant tree size and threads, number of threads with constant query count and tree size, variable tree size (number of points) with constant query count, dimensionality, and threads, and then variable dimensions with constant query count, threads, and tree size.

In test 2, batched range queries are run and the variables tested are number of range queries with constant tree size and threads, number of threads with constant query count and tree size, variable tree size (number of points) with constant query count, dimensionality, and threads, and then variable dimensions with constant query count, threads, and tree size.

In test 3, tree construction time is measured and the variables that are in question are time versus varying number of threads, varying number of points and varying dimensionality with constant number of points and number of threads.

Lastly, test 4 measures the amount of time to insert and delete a number of points. It should be noted that this is not a benchmark for parallelization, rather a test of usability to show the dynamic capabilities of the AST-Flat KD-tree in addition to the other functions.

Each of the four tests was designed to evaluate an aspect of KD-tree usage in real-world parallel computing applications. Test 1 (k-Nearest Neighbor search) targets a computation-intensive workload where the cost of repeatedly calculating distances and maintaining nearest neighbor heaps grows with both the number of queries and dimensionality. By testing this under varying conditions—such as the number of queries, threads, data points, and dimensions—the tree’s ability to scale under high computational loads is evaluated. It is also observed how well the parallel and AST-Flat variants accelerate workloads compared to the sequential baseline. Test 2 (range queries) focuses on a hybrid memory-computation workload, which is essential for spatial filtering, density estimation (like in DBSCAN), and real-time sensing. These queries test both the traversal and memory access patterns, making them useful to assess cache efficiency, thread scaling, and search-space pruning capabilities of the tree structures. Test 3 (construction timing) evaluates communication and memory bandwidth by building trees with various thread counts, data sizes, and dimensions. Since tree construction involves recursive partitioning, memory allocation, and potential data movement between threads, it serves as a

good test of parallel overhead and the efficiency of task scheduling strategies, especially in shared memory systems. Finally, Test 4 (insertion and deletion) tests the dynamic usability and adaptability of each tree variant, that being the basic KD-Tree and the modified version. While not parallelized, this test is useful for applications involving online learning, streaming data, or environments that require incremental updates to the tree without complete reconstruction.

NOTE: All time is measured in seconds

## Experiments and Analysis

### Experiments

#### Test 1 (Batched k-NN Benchmark)

Time vs Number of k-NN queries with Constant Tree Size (100,000), Dimensions (10) and Threads (4)

|            | 1        | 50       | 100     | 500      | 5000     |
|------------|----------|----------|---------|----------|----------|
| Sequential | 0.002224 | 0.055915 | .116927 | 1.286059 | 6.044492 |
| Parallel   | 0.006334 | 0.046664 | .068487 | 0.513622 | 2.588069 |
| AST-Flat   | 0.005567 | 0.014945 | .024759 | 0.120447 | 0.579995 |

Time vs Number of Threads with Constant Tree Size (100,000), Dimensions (10) and queries (500)

|            | 1        | 2        | 4        | 8        | 16       |
|------------|----------|----------|----------|----------|----------|
| Sequential | 0.720832 | 0.762474 | 0.613459 | 0.606061 | 0.700535 |
| Parallel   | 0.847893 | 0.999636 | 0.331313 | 0.214205 | 0.230712 |
| AST-Flat   | 0.159344 | 0.103868 | 0.056619 | 0.066010 | 0.057378 |

Time vs Tree Size with Constant Queries (500), Dimensions (10) and Threads (4)

|            | 50       | 1000     | 50,000   | 100,000  | 1,000,000 |
|------------|----------|----------|----------|----------|-----------|
| Sequential | 0.002466 | 0.082216 | 0.508327 | 0.812727 | 1.298459  |
| Parallel   | 0.004739 | 0.038087 | 0.234949 | 0.314202 | 0.713814  |
| AST-Flat   | 0.005256 | 0.015249 | 0.060141 | 0.089766 | 0.179275  |

Time vs Dimensions with Constant Tree Size (100,000), Queries (500) and Threads (4)

|            | 1        | 3        | 6        | 10       | 20       |
|------------|----------|----------|----------|----------|----------|
| Sequential | 0.000824 | 0.003321 | 0.032915 | 0.617665 | 7.822825 |
| Parallel   | 0.001646 | 0.004995 | 0.036647 | 0.281566 | 3.735345 |
| AST-Flat   | 0.001708 | 0.005318 | 0.020039 | 0.094937 | 0.763634 |

### Test 2 (Batched Range Search Benchmark)

Time vs Number of Range queries with Constant Tree Size (100,000), Dimensions (10) and Threads (4)

|            | 1        | 50       | 100      | 500      | 5000      |
|------------|----------|----------|----------|----------|-----------|
| Sequential | 0.002904 | 0.202122 | 0.321137 | 2.248381 | 19.491016 |
| Parallel   | 0.015750 | 0.197207 | 0.213487 | 0.916284 | 6.910382  |
| AST-Flat   | 0.012878 | 0.205291 | 0.427842 | 1.612269 | 9.808697  |

Time vs Number of Threads with Constant Tree Size (100,000), Dimensions (10) and queries (500)

|            | 1        | 2        | 4        | 8        | 16       |
|------------|----------|----------|----------|----------|----------|
| Sequential | 2.154835 | 1.631315 | 2.181453 | 1.618891 | 1.764922 |
| Parallel   | 2.360051 | 1.455920 | 0.690076 | 0.554602 | 0.466587 |
| AST-Flat   | 3.122486 | 3.220497 | 1.474864 | 0.888749 | 0.634251 |

Time vs Tree Size with Constant Queries (500), Dimensions (10) and Threads (4)

|            | 50       | 1000     | 50,000   | 100,000  | 1,000,000 |
|------------|----------|----------|----------|----------|-----------|
| Sequential | 0.000542 | 0.018445 | 0.681352 | 1.621567 | 15.850925 |
| Parallel   | 0.017130 | 0.015807 | 0.125828 | 0.346999 | 4.603626  |
| AST-Flat   | 0.019549 | 0.010997 | 0.282556 | 0.543015 | 5.656715  |

Time vs Dimensions with Constant Tree Size (100,000), Queries (500) and Threads (4)

|            | 1        | 3        | 6        | 10       | 20       |
|------------|----------|----------|----------|----------|----------|
| Sequential | 1.292171 | 1.032756 | 1.475522 | 1.725222 | 2.581455 |
| Parallel   | 0.182019 | 0.186144 | 0.257763 | 0.255307 | 0.608373 |
| AST-Flat   | 0.423417 | 0.470527 | 0.499338 | 0.506563 | 0.420675 |

### Test 3 (Tree Construction Benchmark)

Time vs Number of Threads with with Constant Size (100,000) and Dimensions (10)

|            | 1        | 2        | 4        | 8        | 16       |
|------------|----------|----------|----------|----------|----------|
| Sequential | 0.484044 | 0.437586 | 0.480871 | 0.426329 | 0.417683 |
| Parallel   | 0.447764 | 0.330383 | 0.306086 | 0.330755 | 0.383259 |
| AST-Flat   | 0.201607 | 0.329261 | 0.304923 | 0.318901 | 0.300421 |

Time vs Size with with Constant Threads (4) and Dimensions (10)

|            | 50       | 1000     | 50,000   | 100,000  | 1,000,000 |
|------------|----------|----------|----------|----------|-----------|
| Sequential | 0.000239 | 0.002520 | 0.307644 | 0.669308 | 6.895842  |
| Parallel   | 0.028719 | 0.038341 | 0.168019 | 0.303573 | 3.441736  |
| AST-Flat   | 0.010948 | 0.036214 | 0.183032 | 0.347286 | 3.628668  |

Time vs Dimensions with with Constant Threads (4) and Size (100,000)

|            | 1        | 3        | 6        | 10       | 20       |
|------------|----------|----------|----------|----------|----------|
| Sequential | 0.252919 | 0.416717 | 0.407955 | 0.418052 | 0.471499 |
| Parallel   | 0.179496 | 0.270920 | 0.277397 | 0.328401 | 0.374168 |
| AST-Flat   | 0.230999 | 0.268989 | 0.310311 | 0.309181 | 0.383878 |

### Test 4 (Insertion and Deletion Benchmark)

Time vs Number of Points to Insert/Delete with Constant Dimensions (10)

|          | 10      | 500     | 50,000  | 100,000 | 1,000,000 |
|----------|---------|---------|---------|---------|-----------|
| Normal   | .000028 | .000807 | .307982 | .725990 | 8.968653  |
| AST-Flat | .000025 | .001010 | .294455 | .951928 | 8.504750  |

## Analysis

### Test 1 Analysis – Batched k-NN Search Performance

This test evaluates the scalability of batched k-NN search across three implementations—Sequential, Parallel, and AST-Flat—under four different experimental conditions. Each test covers a different type of computational stress: query volume, thread concurrency, dataset size, and feature dimensionality.

#### 1. Time vs Number of k-NN Queries

With constant tree size, dimensionality, and thread count, the execution time predictably increases as the number of queries grows. However, the rate of increase is significantly lower for the AST-Flat variant. For example, at 5,000 queries, AST-Flat completes in 0.58s, while the sequential baseline requires 6.04s, resulting in a  $>10\times$  speedup. This is because AST-Flat can exploit query-level parallelism more effectively than the traditional KD-tree, where cache locality and task division are suboptimal.

#### 2. Time vs Number of Threads

With fixed data and query load, this test highlights parallel scalability. The sequential KD-tree shows minimal variation, as expected, since it does not benefit from added threads. The parallel version improves up to 8 threads but regresses slightly at 16, possibly due to synchronization overhead and diminishing returns on shared-memory systems. AST-Flat consistently outperforms both, achieving its lowest time at 4–8 threads. This suggests AST-Flat benefits from a balanced task granularity, which allows OpenMP to efficiently distribute subtree work without excessive task management. This seems to parallel that of the traditional parallel model which stalls in speedup from 8-16 threads while AST-Flat stalls earlier perhaps due to overhead since the times measured are significantly shorter.

#### 3. Time vs Tree Size

Increasing the dataset size affects both the tree traversal and the volume of data touched per query. While all variants experience increased time, AST-Flat remains the most scalable, maintaining under 0.18s even for 1,000,000 points, compared to 1.29s for the sequential version. The consistent performance scaling for all of them implies AST-Flat benefits from flattened median partitioning, leading to shallower tree depths and reduced traversal cost. The sequential implementation, in contrast, faces deeper trees and redundant comparisons leading to slower times and this can be seen in how even though parallelized, it still runs significantly slower than AST-Flat at .7 seconds.

#### 4. Time vs Dimensionality

This test stresses the curse of dimensionality. As dimensionality grows, all variants experience performance degradation, but to varying degrees. The sequential KD-tree suffers the most, with a sharp jump from 0.6s (10

dims) to ~7.8s (20 dims), because hyperplane splits become less effective, resulting in more exhaustive comparisons. AST-Flat shows less degradation, completing in ~0.76s for 20D, highlighting its ability to better handle high-dimensional projections via its random projection matrix, reducing effective search space.

## 5. Final Results

AST-Flat consistently outperforms other variants in terms of timing, demonstrating usage of parallelization and projection-aware search behavior. The parallel KD-tree also sees significant improvement in time; however with the augmentations that are present in the AST-Flat, it is still slower. Dimensionality has the greatest negative impact on all methods, but AST-Flat is more robust due to reduced reliance on directly using the high dimensional points. These findings show that for large-scale or high-dimensional nearest neighbor search, the AST-Flat KD-tree is a strong candidate in terms of performance.

### Test 2 Analysis – Batched Range Query Performance

This test benchmarks the efficiency of executing  $\epsilon$ -neighborhood (range) queries on KD-tree variants under varying loads. These queries mimic the preprocessing steps in density-based clustering algorithms like DBSCAN, where each query returns all points within a defined radius.

#### 1. Time vs Number of Range Queries

With fixed data size and thread count, increasing the number of range queries simulates a growing analytical workload. The sequential one seems to scale linearly and quickly becomes very costly at high query counts due to lack of parallelism. The parallel one scales much better, especially from 100 to 5000 queries. Good use of OpenMP shows substantial improvement. AST-Flat also scales well but becomes less competitive at larger batches. It seems as though the normal KD-tree excels at higher query parallelism because of simpler traversal and more uniform work per thread. AST-Flat incurs more overhead per node due to projection, which accumulates with high query volume

#### 2. Time vs Number of Threads

When scaling threads with a fixed workload, we see that Sequential KD-tree shows flat timings, which is expected since it doesn't utilize threads. Parallel and AST-Flat both benefit up to 4 threads, after which performance slightly worsens or plateaus, particularly for the Parallel variant (e.g., time increases at 8–16 threads). This suggests diminishing returns due to false sharing, NUMA effects, or task management overhead. AST-Flat shows better thread scalability since it does not seem to plateau as earlier however the extra projections and slightly more branching logic limits gains from the parallelism

#### 3. Time vs Tree Size



As tree size increases (up to 1,000,000 points), the Sequential tree still performs linearly. However, the parallel tree's performance sees significant improvement vs sequential as size grows. This is similar to the AST-Flat tree where the numbers show it is slower than Parallel but still scales efficiently. As trees grow, traversal without parallelism in populated shallow layers such as in sequential becomes very expensive. Parallel/AST-Flat benefit from batched traversal, but AST-Flat again incurs cost per node (projection, larger structs).

#### 4. Time vs Dimensionality

Dimensionality is a key stressor for range queries, as volume grows exponentially, and splits become less meaningful. For all of them, They seem to increase with the times between 1 and 3 being fairly negligible. Starting from 6-10, all of them increase in time with the traditional parallel KD-tree being the fastest by a large margin due to the sequential tests' lack of parallelism and overhead from AST-Flat. However, we see at 20 dimensions where the projection starts to excel by bringing down the effective dimensions to a lower limit which reduces the amount needed to calculate.

#### 5. Final Results

The curse of dimensionality affects all variants, but AST-Flat mitigates it best due to projection-based splits, resulting in more meaningful partitions. We see for the number of range queries that times all relatively increase and for the time vs tree size chart as well. While the parallel does excel in many cases, for high dimensional large amounts of data, AST-Flat significantly outperforms the normalized parallel algorithm and the sequential algorithm.

### **Test 3 Analysis – Tree Construction Performance**

#### 1. Time vs Number of Threads

The sequential version shows constant time across threads (as expected). The parallel version sees clear speedup from 1 to 4 threads, but then plateaus then slightly worsens. AST-Flat shows best performance at low thread counts (1–4), but similar plateauing beyond that. This behavior shows diminishing returns on parallelism as threads increase. This is likely due to: thread overhead (scheduling, task splitting) surpassing computational benefit at higher thread counts.

#### 2. Time vs Tree Size

All methods scale non-linearly with tree size, as expected. The sequential KD-tree grows slowly up to 1,000 but drastically slows down for 1,000,000 points. Both parallel and AST-Flat trees handle larger sizes much better. This shows the core benefit of parallelism where until a certain depth (where the more shallow nodes are accessed more often), the more populated nodes are parallelized meaning that they would be inserted more efficiently. While they all benefit from its recursive median-partitioning strategy, the parallelized versions show significant speedup at large sizes since somewhere between 1000 and 50,000 points sequential is slower.

### 3. Time vs Dimensionality

All three methods increase slightly in time with higher dimensions, even up to 20 dimensions. AST-Flat and parallel KD-tree maintain relatively flat performance curves. This may suggest: with either method of sectioning whether it's the flat version or axis cycling, time does not significantly degrade with dimension. Memory usage increases with dimensionality, but if cache lines are efficiently accessed, the penalty is mild. This suggests good implementation efficiency — but for much higher dimensions (50+), there may be larger effects.

### 4. Final Results

AST-Flat is scalable and makes effective use of parallelism. Parallelism gives substantial speedups up to a point, after which overhead dominates. Tree construction of AST-Flat is slightly slower than parallel traditional KD-trees which can be attributed to the larger node structure and associated memory overhead (original point vs original point + projected version of point)

#### Test 4 Analysis – Insertion/Deletion Performance

##### 1. Time vs Number of Points

At all scales, both implementations have negligible differences. At different points, either could be faster with the normal version being slightly faster in the 500 and 100,000 cases so there is no obvious indication which one is faster since both methods have the same insertion and deletion mechanism. Therefore, it is not intended for dynamic updates (insertion/deletion) to be the strength of AST-Flat since it's fundamentally a batch-optimized tree built for static or semi-static datasets, it scales as well as a traditional KD-tree.

#### Accuracy

k-NN test - Average distance to 3-NN vs dimensions (100,000 points)

|              | 1 | 3      | 6       | 10      | 20      |
|--------------|---|--------|---------|---------|---------|
| Trad KD-Tree | 0 | 1.5051 | 12.0575 | 32.1922 | 81.4069 |
| AST-Flat     | 0 | 1.5060 | 12.0609 | 32.2630 | 81.6995 |

Range test - Num points detected vs dimensions (100,000 points)

|              | 1     | 3     | 6     | 10  | 20 |
|--------------|-------|-------|-------|-----|----|
| Trad KD-Tree | 36184 | 47208 | 23663 | 364 | 0  |
| AST-Flat     | 36184 | 47208 | 23663 | 364 | 0  |

## **Conclusions**

AST-Flat KD-tree offers the best scalability for large-scale batched queries at high dimensions (k-NN and range), significantly outperforming both sequential and traditional parallel implementations as query count, point size, or thread count increases without a significant drop in accuracy at the cost of slight increase in performance at lower values of those variables.

Parallelization provides consistent speedups for batched operations and tree construction, but benefits plateau beyond a certain thread count due to memory bandwidth and task overhead — especially for shallow or small datasets.

AST-Flat remains competitive even with dynamic operations (insert/delete), scaling well with large datasets despite not being inherently designed for such use, making it a viable hybrid structure for mixed workloads.

## References

- [1] Tobler, R.F.: The rkd-Tree: An Improved kd-Tree for Fast n-Closest Point Queries in Large Point Sets. In: Proceedings of Computer Graphics International 2011, CGI 2011 (2011)
- [2] X. Zhang, G. Zhang, Improved KD tree high dimensional index algorithm based on location information, *Int. Soc. Optical Eng.* (2020) 11584,.
- [3] Panigrahy R (2008) An improved algorithm finding nearest neighbors using kd-trees. In: Latin American symposium on theoretical informatics. Springer, pp 387–398
- [4] Cao, Y.; Zhang, X.; Duan, B.; Zhao, W.; Wang, H. An improved method to build the KD tree based on presorted results. In Proceedings of the 11th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 16–18 October 2020; pp. 71–75.
- [5] R. Yesantharao, "Parallel batch-dynamic D-trees", 2022.
- [6] Hu L, Nooshabadi S, Ahmadi M. Massively parallel KD-tree construction and nearest neighbor search algorithms. In: 2015 IEEE international symposium on circuits and systems (ISCAS). IEEE; 2015. p. 2752–2755.
- [7] Men, Z., Shen, Z., Gu, Y., & Sun, Y. (2025). Parallel KD-tree with batch updates. *Proceedings of the ACM on Management of Data*, 3(1), 1–26. <https://doi.org/10.1145/3709712>
- [8] Shahnaj, Sabrina, "Construction of Parallel K Dimensional Tree in a Distributed Memory System" (2015). *Theses (Pre-2016)*. 194.